

CHAPTER 2, PART 1

List, ArrayList (Concept, Application)

Key Topics in Chapter 2

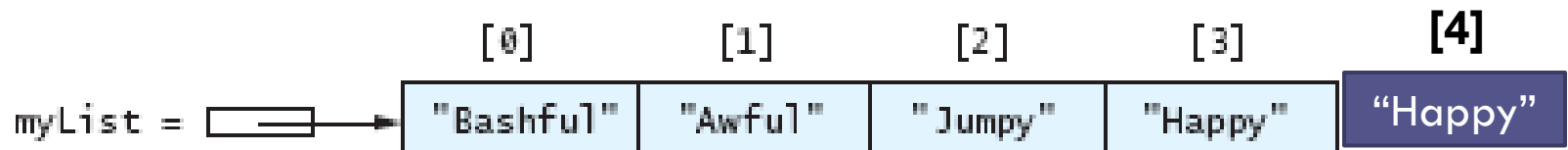
- List data structure
- Array based List data structure
- Linked list data structures:
 - ▣ Singly-linked
 - ▣ Doubly-linked
 - ▣ Circularly-linked

Key Topics in This File

- List data structure
- Java Array
- List interface and its implementing classes: ArrayList, LinkedList
- Generic collections
- ArrayList class
 - Simple usage and applications

List data structure

- A *List*
 - ▣ An **ordered** collection of elements
 - ▣ Each with a **position or index**
 - ▣ Also called **Sequence**
 - ▣ Allow **precise control over where** you want elements to go
 - Position-based access & search
 - ▣ May contain **duplicates**



Java Array

- Is an **indexed** structure
 - ▣ Access elements in any order using subscript values
 - ▣ Access elements in sequence using a loop that increments the subscript
- With the Java `Array` object, you can NOT
 - ▣ increase or decrease its **length** (**fixed**)
 - ▣ **add** an element at a specified position **without shifting** elements to make room
 - ▣ **remove** an element at a specified position and keep the elements contiguous **without shifting** elements to fill in the gap

The List Interface and ArrayList Class

Section 2.1

List Interface in Java API

- List interface

- ▣ In Java API package `java.util`

- ▣ Implemented by

- Abstract class `AbstractList`

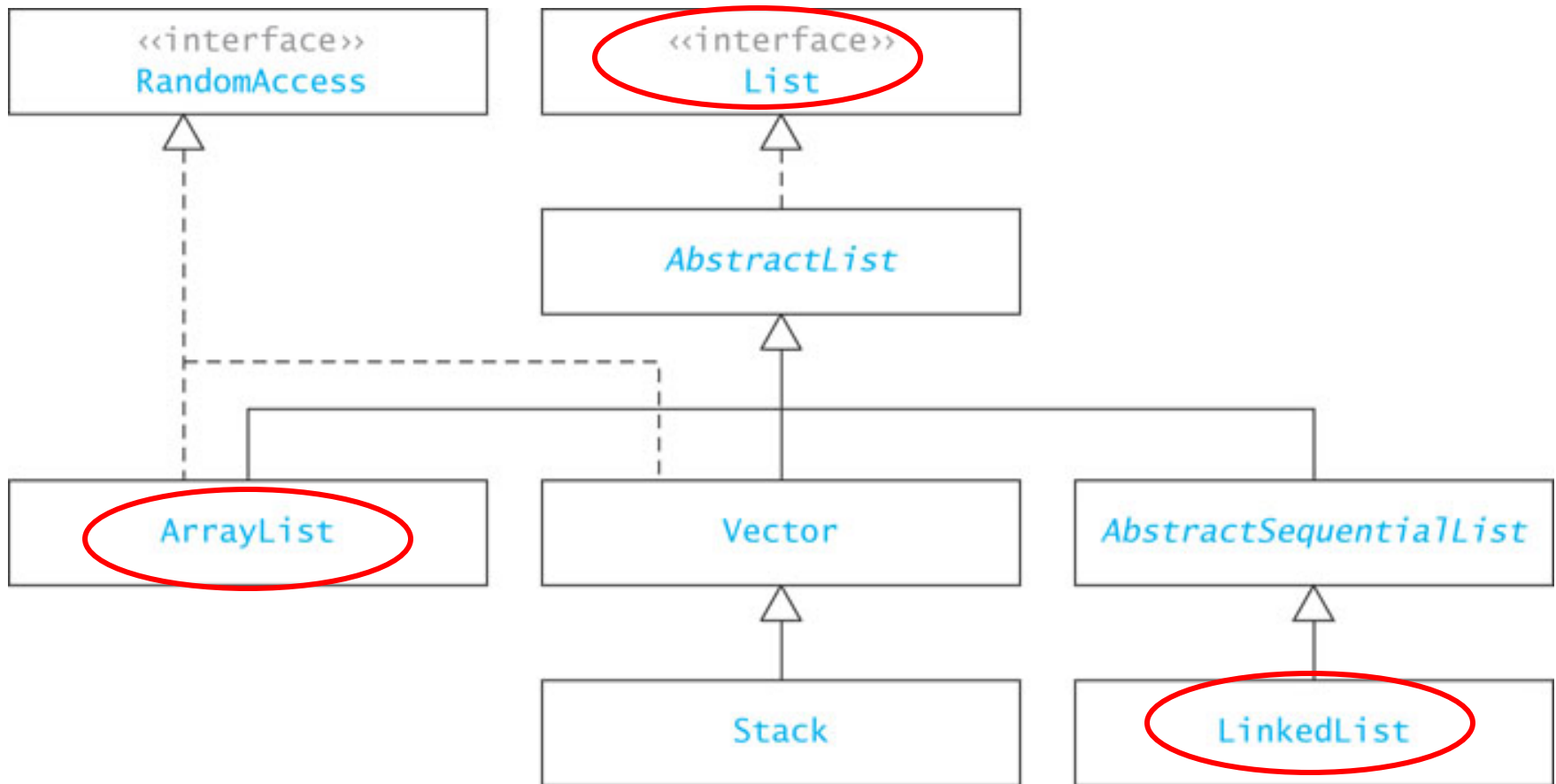
- Concrete Sub classes `ArrayList`, `LinkedList`, `Vector`, etc.

- Classes that implement the List interface

- Provide the functionality of an indexed data structure

- Offer many more operations

java.util.List Interface and its Implementers



List Interface - Operations

- Sample List operations
 - ▣ **Retrieve** an element at a **specified position**
 - ▣ **Replace** an element at a **specified position**
 - ▣ **Find** a specified target **value**
 - ▣ **Add** an element at **either end**
 - ▣ **Remove** an element from **either end**
 - ▣ **Insert or remove** an element at **any position**
 - ▣ **Traverse** the list structure without managing a subscript
- Chapter 2 classes support above operations, but not with same degree of efficiency

ArrayList Class

- Simplest class that implements the List interface
- An improvement over an array object
 - ▣ Dynamic array
- Used when frequently
 - ▣ Add new elements to the end of a list
 - ▣ Randomly retrieve elements based on index

List Interface and ArrayList Class in Java API links

□ Java API Links

▣ **Interface:** `java.util.List`

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html>

▣ **Class:** `java.util.ArrayList`

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html>

Generic Collections – List<String>

```
List<String> myList = new ArrayList<>();
```

- Use a language feature called *generic collections* or *generics*
- Create a List of String;
 - Only String objects can be stored in the list
- **Type argument**
 - String in above statement
 - Set the data type of all objects stored in a collection

Generic Collections - Syntax

General declaration for generic collection:

```
CollectionClassName<E> variable =  
    new CollectionClassName<>();
```

- **type parameter E**

- Takes any specific type like `String`

- **Example**

```
ArrayList<Integer> myList =  
    new ArrayList<>();
```

Generic Collections - Syntax

- Primitive types must be wrapped in objects

```
ArrayList<Integer> myList = new ArrayList<>();  
myList.add(Integer.valueOf(3)); // ok  
myList.add(3); // autoboxing
```

- Adding a **noncompatible type** to a generic collection will generate an **error** during **compile time**

```
ArrayList<Integer> myList = new ArrayList<>();  
myList.add(new String("Hello"));
```

ArrayList Class - Specification

Method	Behavior
<code>public E get(int index)</code>	Returns a reference to the element at position <code>index</code> .
<code>public E set(int index, E anEntry)</code>	Sets the element at position <code>index</code> to reference <code>anEntry</code> . Returns the previous value.
<code>public int size()</code>	Gets the current size of the ArrayList.
<code>public boolean add(E anEntry)</code>	Adds a reference to <code>anEntry</code> at the end of the ArrayList. Always returns <code>true</code> .
<code>public void add(int index, E anEntry)</code>	Adds a reference to <code>anEntry</code> , inserting it before the item at position <code>index</code> .
<code>int indexOf(E target)</code>	Searches for <code>target</code> and returns the position of the first occurrence, or <code>-1</code> if it is not in the ArrayList.
<code>public E remove(int index)</code>	Returns and removes the item at position <code>index</code> and shifts the items that follow it to fill the vacated space.

How to Use ArrayList Class

- ArrayListTest.java
 - ▣ Basic usage of ArrayList of Integer objects, ArrayList of String objects (Using array, List<E>, ArrayList<E>, LinkedList<E>)
- ArrayListTest2.java
 - ▣ Create, populate, and sum up every element in an ArrayList of Integer objects (Using array, ArrayList<E>)
- ArrayListTest3.java
 - ▣ Create, populate, and print every element in an ArrayList of objects of Computer class and its subclasses (Using array, List<E>, ArrayList<E>, LinkedList<E>)
- Use `java.util.List` **interface**, so that the actual list can be either an array list or a linked list.
- Book examples are in next 6 slides.

How to Use ArrayList Class

- Declare a List “object” whose elements will reference String objects

```
List<String> myList = new ArrayList<>();
```

- The initial List
 - Is Empty
 - Has default initial capacity of 10 elements

- Add strings to the list:

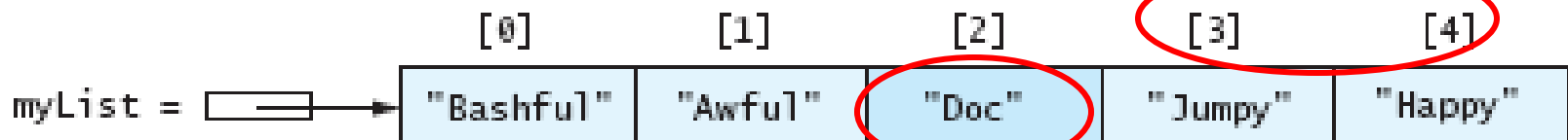
```
myList.add("Bashful");  
myList.add("Awful");  
myList.add("Jumpy");  
myList.add("Happy");
```

How to Use ArrayList Class (cont.)



- Add an element with subscript 2:

```
myList.add(2, "Doc");
```



After insertion of "Doc" before the third element

- NOTE - subscripts of "Jumpy" and "Happy" have changed from [2],[3] to [3],[4]

How to Use ArrayList Class (cont.)

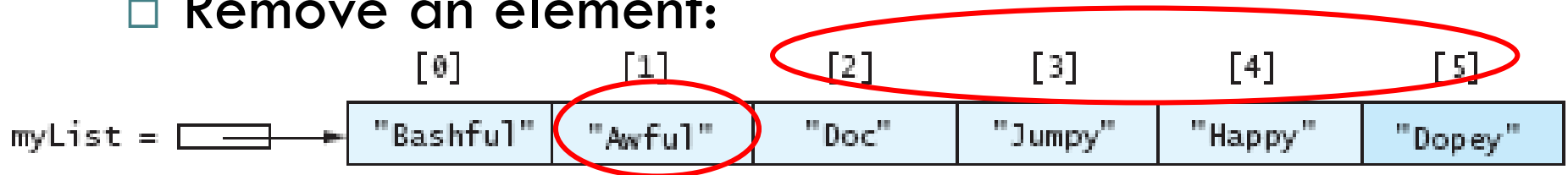
- Add to end of the list (without specified index)

```
myList.add("Dopey");
```

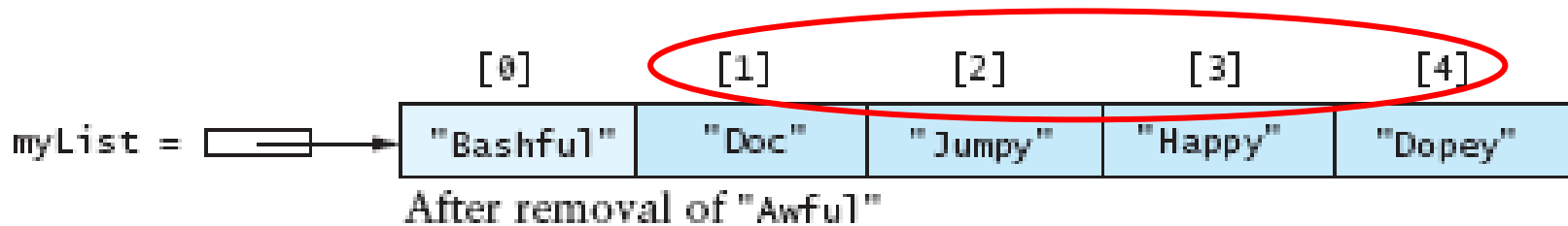


How to Use ArrayList Class (cont.)

□ Remove an element:



```
myList.remove(1);
```



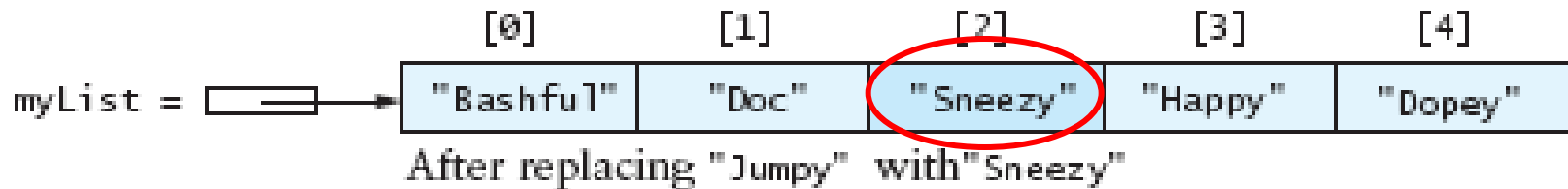
□ NOTE - Indexes of strings from [2] to [5] have changed to [1] to [4]

How to Use ArrayList Class (cont.)

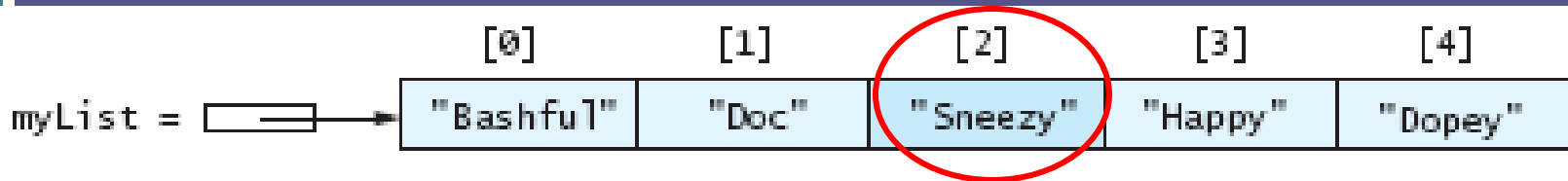
□ Replace an element:



```
myList.set(2, "Sneezy");
```



How to Use ArrayList Class (cont.)

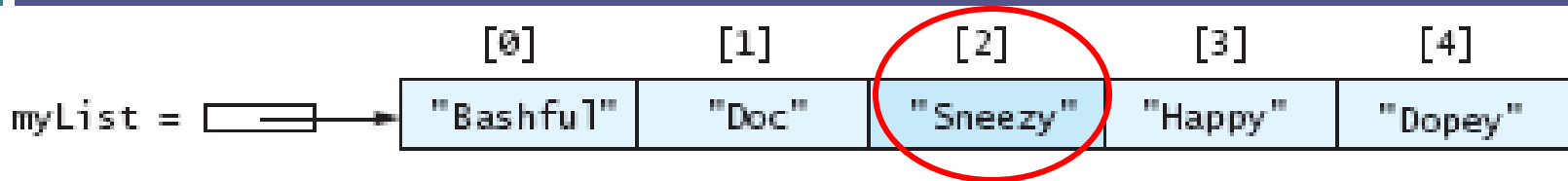


- ❑ Cannot access an element using a bracket index like arrays (`array[1]`)
- ❑ Must use `get()` method:

```
String dwarf = myList.get(2) ;
```

- ❑ The value of `dwarf` becomes "Sneezy"

How to Use ArrayList Class (cont.)



□ Search an ArrayList

□ Search Hit, returns valid index 2

```
myList.indexOf("Sneezy");
```

▣ Search miss, returns -1

```
myList.indexOf("Jumpy");
```

Applications of ArrayList

Section 2.2

Example Application of ArrayList

```
ArrayList<Integer> someInts = new ArrayList<>();  
int[] nums = {5, 7, 2, 15};  
for (int i = 0; i < nums.length; i++) {  
    someInts.add(nums[i]);    //autoboxing  
}  
  
// Display the sum  
int sum = 0;  
for (int i = 0; i < someInts.size(); i++) {  
    sum += someInts.get(i);  
}  
System.out.println("sum is " + sum);
```

Phone Directory Application

- Create a class for Phone Directory Entry

```
public class DirectoryEntry {  
    String name;  
    String number;  
}
```

- Create the phone directory (initially empty)

```
private ArrayList<DirectoryEntry> theDirectory  
    = new ArrayList<>();
```

Phone Directory Application (cont.)

```
public class DirectoryEntry {  
    String name;  
    String number;  
}  
  
private ArrayList<DirectoryEntry> theDirectory =  
    new ArrayList<>();
```

- Add phone directory entry – DirectoryEntry objects

```
theDirectory.add(new DirectoryEntry("Jane Smith",  
                                     "555-1212"));
```

Phone Directory Application (cont.)

- `DirectoryEntry`'s `equals` method compares only name fields.
- `indexOf` searches the `Directory` by applying the `equals` method for class `DirectoryEntry`.

```
int index =  
    theDirectory.indexOf(new DirectoryEntry(aName, ""));  
  
if (index != -1)  
    dE = theDirectory.get(index);  
else  
    dE = null;
```

`equals (Object o)` **method in** **Object Class**

- `equals (Object o)` **method in Object Class**
 - ▣ Header:
 - `public boolean equals (Object other)`
 - ▣ This method **compares reference values** of two objects, and does **NOT compare data contents** in these two objects.
 - ▣ In this strict version, only if the two objects are the exact same object, the method returns true.
 - ▣ Typically a class will override this strict version and provide a new version that compare two objects based on their data contents.
 - ▣ Example – `ObjectEqualTest.java` (not in book)

`equals (Object o)` **method in** `DirectoryEntryV2` **Class**

- `equals (Object o)` **method in**
`DirectoryEntryV2` **Class**
 - ▣ **Header:**
 - `public boolean equals (Object other)`
 - ▣ This method overrides the strict version in class `Object` and provides a new version that compares two directory entries only based on the name and does not use phone number in the comparison.
 - ▣ Example Use – `ObjectEqualTest.java`